



Research



Cite this article: Harrison JM, Yeung E. 2026 Estimating varying parameters in dynamical systems: a modular framework using switch detection, optimization and sparse regression. *R. Soc. Open Sci.* **13**: 251250. <https://doi.org/10.1098/rsos.251250>

Received: 8 July 2025

Accepted: 5 January 2026

Subject Category:

Mathematics

Subject Areas:

applied mathematics

Keywords:

parameter estimation, parameter-varying systems, input estimation, switch detection, optimization, regression

Author for correspondence:

Jamiree M. Harrison
e-mail: jamiree@ucsb.edu

Estimating varying parameters in dynamical systems: a modular framework using switch detection, optimization and sparse regression

Jamiree M. Harrison and Enoch Yeung

Department of Mechanical Engineering, University of California Santa Barbara, Santa Barbara, CA, USA

JMH, 0000-0002-6535-1579; EY, 0000-0001-7630-7429

The estimation of static parameters in dynamical systems is well-established, but challenges remain for systems with parameters that vary over time or space. Assuming prior knowledge of the system's dynamics, we aim to identify functions describing these varying parameters. Initially, we focus on cases where parameters are discretely switching piecewise constant functions. For this sub-class, we propose an algorithmic framework to detect discrete parameter switches and fit a piecewise constant model using optimization-based parameter estimation. Our framework is modular, allowing customization of switch detection, numerical integration and optimization steps. We demonstrate its utility through examples, including a time-varying promoter-gene expression model, a genetic toggle switch, parameter-switching manifolds, mixed fixed and switching parameters, non-uniform switching and models governed by PDEs like the heat and advection–diffusion equations. Binary segmentation is used for switch detection, and Nelder–Mead and Powell methods for optimization. In addition, we assess the framework's robustness to measurement noise and time-series data resolution. For more complex cases, we extend the approach using dictionary-based sparse regression with trigonometric and polynomial functions to capture continuously varying parameters. We also compare our framework with two other similar methods using standard metrics under the condition of model-mismatch. This comprehensive framework offers a versatile solution for parameter estimation in varying dynamical systems.

1. Introduction

Parameter estimation is a fundamental inverse problem which finds applications in control theory, statistics, systems biology and physics, to name just a few. Estimating a system's parameters often precedes the prediction of that system's future states, the understanding of that system's dynamics, and the structure of the given system. Obtaining a system's parameters in certain instances is a key step in the accurate control of that system. To exemplify the estimation of a system's parameters for its accurate control, the work of Ryan & Ichikawa [1] shows how this can be done in the context of a linear distributed system with static parameters that get updated estimates. Furthermore, parameter estimation has been shown to provide benefits in addressing model discrepancies in control systems. For example, Burns *et al.* [2] address the accurate control of systems with delays, model errors and measurement errors by employing hierarchical modelling to estimate uncertain disturbances, offering an alternative to Bayesian analysis.

Mitra & Hlavacek offer a comprehensive review of parameterization and uncertainty quantification in mathematical models of immunoreceptor signalling and other biological processes. They highlight techniques such as gradient-based estimation, profile likelihood, bootstrapping and Bayesian inference, emphasizing their applications in immune system modelling [3]. Penas *et al.* introduced saCeSS, a novel parallel optimization method for parameter estimation in large-scale kinetic models. This approach demonstrated robust performance, significantly reduced computation times, and user-friendly self-tuning, with potential applications in systems biology and whole-cell dynamic modelling [4].

Zhan & Yeung proposed two innovative parameter estimation methods that combine spline theory with linear and nonlinear programming. These methods offer robust, efficient and flexible solutions for identifying parameters in systems biology models without the need for ODE solvers [5]. Valderrama-Bahamóndez & Fröhlich compared Markov chain Monte Carlo (MCMC) techniques for estimating rate parameters in nonlinear ODE systems used in systems biology. They found that parallel adaptive MCMC, combined with informative Bayesian priors, outperformed other methods for high-dimensional, multi-modal parameter distributions [6]. Giampiccolo *et al.* introduced a hybrid neural ODE framework for parameter estimation and identifiability assessment in biological models with partial structural knowledge. Their approach integrates hyperparameter tuning with *a posteriori* identifiability analysis to address challenges posed by neural networks, demonstrating effectiveness on noisy, real-world-inspired benchmarks [7].

For additional examples in quantitative biology, Ashyraliyev *et al.* provide a mini-review exploring the applications of mathematical models in biology. They focus on parameter inference, model identifiability and commonly used methods for parameter space exploration [8]. Beyond quantitative biology, parameter estimation has been successfully applied in fields like astrophysics. For instance, Douspis *et al.* [9] and Bailer-Jones [10] have utilized parameter estimation across various models and datasets, demonstrating its versatility and effectiveness.

The problem we aim to solve in this paper is at the interface of dynamical systems theory and parameter estimation. Dynamical systems in which the parameters vary as the system evolves are known as parameter-varying systems, and these parameters can change continuously or switch discretely. Systems that are modelled with the prior assumption of having static parameters could often be better characterized, better controlled and more accurately captured via parameter-varying models, and our paper serves as a novel avenue for fitting these more accurate models. An equivalent classification of this set of problems is the input estimation problem because mathematically representing these time-varying parameters as inputs which change over time is indistinguishable as noted in [11]. Accurately estimating the varying parameters along with their switch times for general classes of systems remains a non-trivial task. In this paper, we address this challenge.

1.1. Definitions

We first need to define what constitutes a discrete parameter switch in a parameter-varying system, as well as clarify what it means for a parameter to vary continuously.

Definition 1. Continuously varying parameter.

We say that $p(t) : \mathbb{R} \rightarrow \mathbb{R}$ is a *continuously varying parameter* if $p(t)$ is continuous and not a constant function.

Definition 2. Discrete parameter switch.

Let n_p be defined as the number of parameters in a given system. Suppose that we have piecewise parameters $p(t) : \mathbb{R} \rightarrow \mathbb{R}^{n_p}$, given by

$$p(t) = \begin{cases} p_1(t) & \text{if } t \in [t_0, t_1], \\ p_2(t) & \text{if } t \in (t_1, t_2], \\ \vdots & \\ p_n(t) & \text{if } t \in (t_{n-1}, t_n], \end{cases} \quad (1.1)$$

where $n \in \mathbb{N}$ and $p_i(t)$ are continuously varying. If $p_k(t_k) \neq p_{k+1}(t_k)$ for a given $k \in \{1, \dots, n-1\}$, then we say that a **discrete parameter switch** has occurred at t_k . The number of discrete parameter switches is the **discrete switch number** N_{s_d} . These **discrete switch locations** which we denote by t_{k_d} are jump discontinuities of $p(t)$.

We say that a **continuous parameter switch** occurs at **continuous switch locations** denoted by t_{k_c} at values of t where $p(t)$ is continuous but not differentiable. Explicitly, a continuous switch location occurs if $p_k(t) \neq p_{k+1}(t)$ for some $t \in [t_{k-1}, t_k) \cup (t_k, t_{k+1}]$, while $p_k(t_k) = p_{k+1}(t_k)$. The number of continuous parameter switches is the **continuous switch number** N_{s_c} . The **total switch number** is defined as $N_s := N_{s_c} + N_{s_d}$.

Remark 1. Suppose that we have a dynamical system $\frac{dX}{dt} = f(X, p(t))$, and suppose that f is continuous with respect to t if $p(t)$ is continuous (without discrete switches).

If we have $\frac{dX}{dt} = f(X, p(t))$ such that $p(t)$ represents piecewise parameters with discrete parameter switches at t_k (and thus, jump discontinuities) as described in definition (2), then f has jump discontinuities at t_k and $X(t)$ is non-differentiable at t_k .

The above remark expresses the discrete parameter switches of a parameter-varying system manifest in a given trajectory, $X(t)$, as non-differentiable points in t . In a data-driven empirical setting where we are given discrete data points, X_{data} , we cannot analytically determine where these parameter switch locations are, so we leverage the data-driven switch detection methods that are available in the literature (see §2.2) and we extend their implementations to detect switching parameters of dynamical systems.

1.2. General problem formulation (continuous-time systems)

Assume that we have data denoted $X_{data} \in \mathbb{R}^{m \times N}$ from a parameter-varying system denoted

$$\frac{dX}{dt} = f(X, p(t)), \quad (1.2)$$

with $m \in \mathbb{N}$ states, $N \in \mathbb{N}$ data points, vector field $f : \mathbb{R}^m \rightarrow \mathbb{R}^m$ and $n_p \in \mathbb{N}$ parameters $p(t) : \mathbb{R} \rightarrow \mathbb{R}^{n_p}$.

Further, assume we know the form of the system's model, $f(X, p(t))$ but we do not know the varying parameters $p(t)$. Last, assume that the parameters, $p(t)$, are piecewise functions.

Problem: Identify $p(t)$ given X_{data} , i.e. find the minimizing parameters such that

$$p(t) = \text{argmin} \|X_{data} - X_{model}(p(t))\|_2, \quad (1.3)$$

where $X_{model} \in \mathbb{R}^{m \times N}$ is given by $X_{model} = \int_{\text{numerical}} \frac{dX}{dt} dt$.

Remark 2. Here, the specified numerical integration scheme (ODE solver) is presupposed such that it will grant sufficiently accurate X_{model} data in terms of $\|X_{model} - X(t_d)\|_2 < \epsilon$ for some desired $\epsilon > 0$, where $X(t_d)$ is the corresponding discretization of $X(t)$, the solution to the system. This problem formulation also accounts for PDEs and discrete-time parameter-varying systems are accounted for in the supplement.

This problem formulation encompasses several sub-problems, each addressed with varying degrees of success in the literature. While these contributions demonstrate rigour and specificity in tackling their respective challenges, our framework addresses the general class of problems holistically, filling critical gaps. In addition, we acknowledge that the specialized methods discussed in the literature may outperform our framework in certain domains and should be employed when appropriate. However, our approach effectively covers many use cases within the largely unexplored space of the broader problem class.

For example, Wang *et al.* proposed a method for switch detection and robust identification in slowly switched Hammerstein systems [12]. Zheng *et al.* developed a technique for identifying fast-switched

linear–nonlinear alternating subsystems, particularly in high-speed train models [13]. Yu *et al.* [14] and Bianchi *et al.* [15] introduced randomized methods to address switched autoregressive exogenous and nonlinear autoregressive exogenous systems, single input, single output systems and other non-autonomous systems. Li *et al.* created methods for identifying nonlinear Markov jump systems [16]. Boddupalli *et al.* developed machine learning methods involving symbolic regression to give interpretable representations of dynamical systems without the need of instantiating known models or dictionary functions, but these methods struggle to identify systems with varying parameters [17]. These methods stand in contrast to the recent data-driven system identification and control developments utilizing Koopman operator theory frameworks [18–20].

Kon *et al.* [21] presented an input–output linear parameter-varying (LPV) feedforward parameterization technique and a corresponding data-driven estimation method, leveraging neural networks to model the dependency of coefficients on scheduling signals. Xu *et al.* developed an algorithm for parameter estimation in time-varying systems with invariant matrix structures [22]. Similarly, Souza & Palhares tackled parameter estimation in linear, continuous, time-varying dynamical systems, assuming known coefficient matrices, measurable states and bounded piecewise continuous parameters [23].

Johnson *et al.* [24] applied hybrid gradient descent to estimate unknown constant parameters in hybrid systems with flow and jump dynamics affine in the parameters. Qian *et al.* used Bayesian parameter estimation for discrete-time linear parameter-varying finite impulse response systems [25]. Altaie *et al.* [26] demonstrated how parameter estimation in parameter-varying systems can inform the design of robust controllers for multidimensional systems.

Further, we showcase implementations of algorithms within our framework that successfully handle several examples including a time-varying promoter–gene expression model, the time-varying genetic toggle switch, a parameter-switching manifold, a model with a mixture of fixed and switching parameters, a model in which parameters are not switching uniformly and the advection–diffusion equation with a time-varying coefficient of advection. Our paper contributes to the body of literature in that it provides a framework to estimate the changing parameters of parameter varying dynamical systems using an approach that utilizes the combination of switch detection, optimization and sparse dictionary regression.

1.3. Contributions

- (1) Ensemble methodological framework that allows for estimation of time-varying parameters/inputs for the broad class of parameter/input estimation problems.
- (2) Precise switch locations of parameter/input estimates obtained from data, allowing for accurate parameter/input based data segmentation.
- (3) Function parameterizations of time-varying parameter/input estimates are obtained from data with sparse regression.
- (4) Adapted signal change-point detection algorithms to detect parameter/input switches in dynamical systems.

2. Methodology for algorithmic framework

To give a broad overview of our framework, let us define $X_{data} \in \mathbb{R}^{m \times N}$ as the input data from our system of interest, and $X_{model} \in \mathbb{R}^{m \times N}$ as the predictive model data that approximates X_{data} upon completion of an algorithm within our framework. These are matrices with m being the number of states and N being the number of data points (or snapshots) such that a row of either X_{data} or X_{model} is a state trajectory. A block diagram overview of our framework is given in figure 1 and an outline is given in figure 2.

The framework in figure 1 starts with the initialized continuous-time model, then detects switches in the input data and then segments the data into n intervals. From here, model parameters $p(t)$ and p_{static} are identified to fit the model to the input data over each segmented time interval. This is done by numerically integrating the segmented dynamic model to obtain X_{model_i} and then using numerical optimization to minimize the distance (norm) between X_{data_i} and X_{model_i} , where $i \in \{1, \dots, n\}$. Numerical integration is used for differential equation models in the form of $\frac{dX}{dt} = f(X, p(t))$ and is not used for parametric curve models in the form of $X_{model} = f(X, p(t))$. If the parameters were presupposed to be piecewise constant, then our algorithm outputs them as such. If the parameters were presupposed to be continuously varying functions, then we assume a high number of switches to have occurred in the data and use

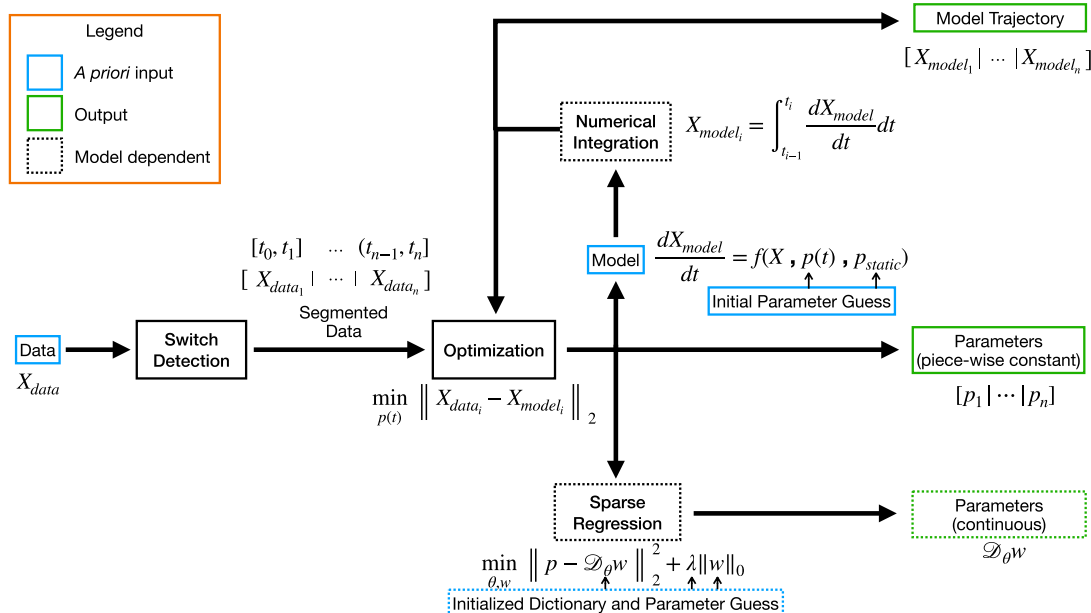


Figure 1. Outline of algorithmic framework. The data (left) get segmented by a switch detection algorithm. The presupposed model (middle-right) is then fitted to each data interval via minimization of the model trajectory error (middle-left). If the model is a differential equation, then the numerical integration substep (upper-middle) must take place in the optimization loop (middle), but numerical integration need not take place if the model is a parametric equation in the form of $X_{model} = f(X, p(t))$. Optimal parameters and model trajectories are then given as output (right). Sparse regression is used if the parameters of the model are pre-supposed to be continuously varying as opposed to being discretely switching piecewise constant parameters.

dictionary-based sparse regression to fit continuous functions to the parameters. Finally, the identified model parameters are used to generate the model trajectory, X_{model} , that accurately approximates X_{data} . Being that this trajectory is an approximation of the N sequential snapshots given in X_{data} , these are also known as N -step predictions.

From here it is important to note that this framework works for ODEs and PDEs that are written to have a first-order derivative in time. In the context of numerical methods for ODE solving, this means that the ODE must be in standard form. One can handle PDEs within our framework using the method of lines (discretizing the spatial derivative) [27]. Further, we showcase and provide code in the supplement for our given implementations, but it should not be understated that given the modular structure of the code, that a user can swap out the switch detection, optimization and numerical integration schemes for preferred methods. Further, our framework is easily modulated to estimate the varying parameters of a discrete time system (found in supplement).

Remark 3. For globally optimal solutions to be granted under this framework (over finite domains), all substeps listed in the framework must be globally optimal. If global optimization algorithms are used over a finite domain of possible parameters, then globally optimal parameters are guaranteed over that domain; however, these globally optimal parameters may not be unique because other distinct minima of the specified non-convex objective function, $\|X_{model} - X_{data}\|_2$, may exist.

2.1. Evaluative metrics

Now that we have established our algorithmic framework, we formalize the metrics to be used for the evaluation of such algorithms. The parameter error E_p is defined by the distance between a system's set of true parameters over the discrete data points, $p(t_d)$, and the approximate set of parameters of that system, $\hat{p}$, normalized by the number of parameters, n_p . The trajectory error E_t is defined by the distance between the input data matrix, X_{data} , and the data matrix generated by the given model with estimated parameters, X_{model} , normalized by the number of data points N . From context, it is clear that the trajectory error E_t is the objective function minimized in our framework. The switch number error E_{N_s} is the difference between the true number of switches, N_s , and those detected, $\hat{N}_s$. Note that if E_{N_s} is positive,

1. Initialize: model, parameter guesses, and $X_{data} \in \mathbb{R}^{m \times N}$

$$\frac{dX_{model}}{dt} = f(X, p(t), p_{static})$$

2. Segment data into $n := N/6$ intervals using switch detection

$$\begin{array}{cccccc} [t_0, t_1] & (t_1, t_2] & \dots & (t_{i-1}, t_i] & \dots & (t_{n-1}, t_n] \\ X_{data} = [& X_{data_1} & | & X_{data_2} & | & \dots & | & X_{data_i} & | & \dots & X_{data_n}] \\ p = [& p_1 & | & p_2 & | & \dots & | & p_i & | & \dots & p_n] \end{array}$$

3. Minimize objective function to obtain parameters for $[t_0, t_1]$

→ **Numerically integrate:** $X_{model_1} = \int_{t_0}^{t_1} \frac{dX_{model_1}}{dt} dt$

→ **Optimize:** $\{p_1, p_{static}\} = \text{argmin} \left\| X_{data_1} - X_{model_1} \right\|_2$

4. Iterate for the $i = 2, \dots, n$ segmented data intervals assuming p_{static} is known

↻ **Optimize:** $\{p_i\} = \text{argmin} \left\| X_{data_i} - X_{model_i} \right\|_2$

5. Sparse regression on p

→ **Optimize:** $\{\theta, w\} = \text{argmin} \left\| p - \mathcal{D}_\theta w \right\|_2^2 + \lambda \|w\|_0$

Figure 2. Outline of algorithm with sparse regression for the identification of continuously varying parameters.

then the number of switches detected is an underestimate and if E_{N_s} is negative, then the number of switches detected is an overestimate. In this way, the sign of E_{N_s} can be used to quantify if a given algorithm systematically under-reports or over-reports the correct number of switches in data. To measure the distance from the true switches, $\{t_1, \dots, t_{n-1}\}$, to the detected switch estimates, $\{\hat{t}_1, \dots, \hat{t}_{\hat{n}-1}\}$, we use the Hausdorff metric, H_s which measures the worst error among all of the detected switches and is explicitly given by

$$H_s := \max\{\max_k \min_l |t_k - \hat{t}_l|, \max_{\hat{k}} \min_l |\hat{t}_{\hat{k}} - t_l|\}. \quad (2.1)$$

Remark 4. For explicit clarity, the following would generally be unknown in a practical scenario before any algorithm is applied: the true parameters $p(t_d)$, the true number of switches N_s and the locations of the switches t_k . The point of our framework and the goal of solving the previously stated problem is to infer what these unknowns are. We provide these metrics so that algorithms suited to our stated problem-class can have their performances benchmarked when applied to datasets and systems for which $p(t_d)$, N_s and t_k are known.

We now briefly explain our choice of the switch number error E_{N_s} and the Hausdorff metric H_s over other metrics commonly reviewed in the literature [28]. The precision and recall metrics, for instance, declare a change point as ‘detected’ if at least one computed change point is within a ‘margin’ of data points from it [28]. However, these metrics require the introduction of an additional hyperparameter, margin, which is undesirable for our use case, as it already involves a significant number of user-defined parameters. Another alternative is the Rand index, which measures the similarity between two segmentations as the proportion of agreement between their partitions [29]. While this metric has merit, we chose the Hausdorff metric over the Rand index because we find the distance between true and detected switches to be a more intuitive and interpretable measure than a similarity score between the sets of true and

detected switches. Ultimately, we recognize that the choice of metrics is inherently subjective. Nevertheless, we assert that E_{N_s} , in combination with H_s , provides an effective and practical means of evaluating errors in our specific context.

2.2. Detection of parameter switches in data

Switch detection (commonly referred to as change point detection) in noisy signals remains challenging, and the optimal method for a given dataset depends on the signal type, noise level and number of expected changes. One popular method, Bayesian online change point detection (BOCPD), employs a Bayesian framework to sequentially update the probability of change points. It excels in handling noisy signals by incorporating noise into the model, making it useful for online detection in streaming data [30]. However, BOCPD can be computationally intensive and sensitive to prior assumptions. Pruned exact linear time (PELT), another widely used method, minimizes a cost function plus a penalty term to avoid over-fitting to noise, offering scalability to large datasets and flexibility in cost function choice. Its performance depends on a penalty term, making it suitable for large datasets with unknown change points [31]. For signals with high noise levels, total variation regularization (TVR) effectively detects change points while simultaneously denoising, though it may smooth out small changes [32]. Cumulative sum detects mean shifts by monitoring the cumulative sum of deviations from a reference point, with robust variants for noisy data, though it is less effective for variance or distributional changes [33]. Dynamic programming (exact segmentation) offers an exact solution by minimizing the cost function over all possible segmentations but is computationally expensive and requires the number of switches to be known *a priori*, making it more suitable for offline detection [34]. Hidden Markov models (HMMs) are ideal for time series with hidden states and can model transitions as change points, though selecting the number of hidden states and computational cost are concerns [35]. Kernel-based methods, which map data to high-dimensional spaces, excel at detecting complex, nonlinear changes but require parameter tuning and can be computationally intensive [36]. Finally, wavelet-based methods analyse signals at multiple scales, naturally denoising while detecting change points, making them suitable for noisy data with multiscale structures. The choice of method depends on the type of change, noise level and whether online or offline detection is needed. For low-to-moderate noise, PELT or BOCPD are recommended. For high noise, TVR or wavelet-based methods work well, while for complex signals, kernel methods or HMMs are preferable.

In the field of switch detection, different cost functions are used to tailor the detection method to the specific characteristics of the data. L_1 and L_2 cost functions are basic measures used for minimizing absolute and squared residuals, respectively, where L_1 is more robust to outliers, while L_2 is sensitive to large deviations. The normal cost from [31] assumes data follows a normal distribution and detects changes in mean and variance. More complex cost functions using radial basis functions (RBFs) and sinusoids detect changes in data patterns by comparing similarity measures, with RBFs suitable for nonlinear changes, and sinusoids focusing on angular shifts in data [37]. Cost functions for data with linear relationships and continuous linear relationships account for gradual linear changes over time [36]. Cost functions which rank data values are robust to non-Gaussian noise [38]. Maximum likelihood cost functions adapt to various statistical models for flexible change detection [39]. Finally, auto-regressive cost functions identify changes in time-series data with auto correlation by capturing shifts in time dependencies [40]. Every cost function mentioned can be selected based on the specific structure and noise characteristics of the data.

For the implementation of switch detection in our framework for the examples in this paper, we used binary segmentation (`binseg`) which is a recursive algorithm that was devised to detect switches in noisy data [41]. The algorithm is sequential in nature, in that it first detects one change point, or switching point, in a given input data signal, then it splits the data at this change point, and this process is recursively repeated on the two resulting sub-signals until no more change points are detected in all sub-signals [42].

Our implementation of `binseg` uses two hyperparameters, where SD_{algo} below is `binseg` in our examples.

$$\sigma = \begin{cases} \text{standard deviation of noise,} & \text{if number of switches, } N_s, \text{ is unknown,} \\ \text{number of switches } N_s, & \text{if number of switches, } N_s, \text{ is known.} \end{cases} \quad (2.2)$$

$$s_g - \text{switch gap: minimum number of data points between detected switches.} \quad (2.3)$$

Algorithm 1: Switch Detection Over All States of a System's Model

Input: $X_{data} \in \mathbb{R}^{m \times N}$: data matrix

 $t \in \mathbb{R}^N$: independent variable

 $s_g \in \mathbb{N}$: switch gap threshold

 SD_{cost} : switch detection cost function model

 SD_{algo} : switch detection algorithm

 $\sigma > 0$ or $\sigma = N_s \in \mathbb{Z}_{\geq 0}$: noise level or fixed number of switches

Output: $N_s \in \mathbb{Z}_{\geq 0}$: number of switches detected

 $\hat{t}_k$: switch locations (indices of t where $k \in \{1, \dots, N_s\}$)

```

1 Function Switch_Detect( $X_{data}, t, s_g, \sigma, SD_{cost}, SD_{algo}$ ):
2   ( $m, N$ )  $\leftarrow$  shape( $X_{data}$ );
3    $\hat{t}_k \leftarrow \emptyset$ ; // Initialize true switch locations across states
4   for  $j \leftarrow 1$  to  $m$  do
5      $\hat{t}_k \leftarrow SD_{algo}(X_{data}[j, :], SD_{cost}, \sigma)$ ; // Detected and sorted switches for state  $j$ 
6     for  $i \leftarrow 1$  to length( $\hat{t}_k$ ) do
7       if  $\hat{t}_k[i] > \hat{t}_k[i-1] + s_g$  then
8          $\text{result} \leftarrow \text{append}(\hat{t}_k[i])$ ; // Applies  $s_g$  for state  $j$ 
9       if result[end] ==  $N$  then
10         $\text{result} \leftarrow \text{delete}(\text{result}, \text{end})$ ; // Last entry of  $t$  cannot be a switch location
11       $\hat{t}_k \leftarrow \text{result}$ ;
12      for each entry  $\in \hat{t}_k$  do
13        if entry  $\notin \hat{t}_k$  then
14           $\hat{t}_k \leftarrow \text{append}(\text{entry})$ ; // Removes redundant switches across states
15         $\hat{t}_k \leftarrow \text{sort}(\hat{t}_k)$ ;
16  if length( $\hat{t}_k$ ) == 0 then
17     $N_s \leftarrow 0$ ;
18    return  $\hat{t}_k, N_s$ ; // If no switches detected, return
19  if length( $\hat{t}_k$ ) > 0 then
20    for  $i \leftarrow 1$  to length( $\hat{t}_k$ ) do
21      if  $\hat{t}_k[i] > \hat{t}_k[i-1] + s_g$  then
22         $\text{result} \leftarrow \text{append}(\hat{t}_k[i])$ ; // Applies  $s_g$  across states
23       $\hat{t}_k \leftarrow \text{result}$ ;
24  if  $\sigma == N_s$  and  $\sigma \sim \text{length}(\hat{t}_k)$  then
25     $\text{throw error: "detected } N_s \sim \text{expected } N_s"$ 
26   $N_s \leftarrow \text{length}(\hat{t}_k)$ ;
27  return  $\hat{t}_k, N_s$ ;
    
```

Already existing as a feature of `binseg`, the user must input $\sigma > 0$, which is the standard deviation of the noise of the input signal. This noise level can be statistically estimated or inferred by the user as a pre-processing step. Alternatively, the user may also iteratively try different σ values until a desired model accuracy is achieved, or the user can smooth the data with a smoothing algorithm of one's choice such as the Savitzky–Golay filter [43], finite impulse response methods [44] or the Whittaker–Henderson smoother [45]. The lower σ is, the more sensitive the switch detection is, meaning that more switches will typically be detected for a given signal. This is intuitive because `binseg` does not want to mistake noisy ‘jumps’ for a switch in the signal. There is also the case where the user may already know how many switches there are in the given input signal, and does not need `binseg` to estimate the number of switches. If this is the case, the user may specify the number of switches to `binseg` as $\sigma = N_s$ and `binseg` will simply find the switch locations. Note that σ is the most important hyperparameter in determining the number of switches and their locations in given signals.

We denote the ‘switch gap’ as $s_g \in \mathbb{N}$ which does not allow there to be two parameter switches to be within s_g discrete data points from each other. This was implemented to allow some necessary user discretion when applying switch detection methods. To expand, several methods such as `binseg` may detect switches at consecutive time points and at other intervals that are too small for multiple switches to have occurred for certain classes of problems. Simply setting $s_g = 1$ allows for consecutive data points to be detected as switches.

For our implementation of `binseg`, all states (observed input signals) of an ODE or PDE are searched individually for the correct number of parameter switches as opposed to searching over all of the states at once. We do this by running `binseg` on each state's signal, collecting every switch that occurs, and removing switches that have already been counted. For example, suppose we have two states of a system given by x_1 and x_2 . If it is determined that state x_1 experienced two switches at t_1 and t_2 and state x_2 experienced the same two switches, but also two more completely different switches at t_3 and t_4 , then

we say the system has four switches in total, namely, two shared (coupled) switches, and two unique (uncoupled) switches in the second state (for an example of this, see [equation \(3.16\)](#)). This implementation grants the correct number of switches with accuracy when used with optimal switch detection algorithms on systems with discretely switching parameters.

Remark 5. *If one was to naively use a switch detection algorithm directly on X_{data} without the nuances of our implementation, then there would be severe errors E_{N_s} and H_s in several cases such as the example given in §3.3 where the parameters switch at non-uniform intervals. Ultimately, since systems with multiple states can have parameters that switch distinctly and independently across those states, the necessity of our dynamical systems switch detection implementation is clear.*

2.3. Optimization schemes, known static parameters, parameter bounds, constraints and complexity

Upon experimenting with several numerical schemes to minimize our desired objective function our best results came from robust, non-gradient based, deterministic optimization schemes, namely, the Nelder–Mead method [46] and the conjugate-based Powell’s method [47]. Auto-differentiation based methods and stochastic methods could often give comparable results, but not as consistently as the deterministic methods aforementioned. The lack of consistency using auto-differentiation-based methods is probably due to the combination of the general non-convex nature of objective functions and the stochastic elements of the methods. It may be possible to get improved performance with auto-differentiation-based methods or in general from adding regularization to our objective function, but this is yet to be explored. In [48] global optimization schemes are strongly recommended for handling the inherent non-convexity of these data-driven problems, especially in the context of biological systems. With this insight, it is important to note that the modular nature of our framework allows for the implementation of global optimization schemes on finite domains instead of the optimization schemes mentioned previously. Commonly used global optimization methods include basin hopping [49], brute force, differential evolution [50], dual annealing [51], SHGO [52] and the DIRECT algorithm [53]. In the context of parameter estimation and especially when using global optimization methods, the utilization of parameter bounds and constraints is crucial to mitigate the computational complexity of navigating these spaces of parameters. Parameter bounds and constraints can be used within our framework along with the distinctive feature of estimating a subset of parameters as constant. These features serve as ways to help tackle the non-convexity and computational efficiency of these data-driven optimization problems as they reduce the space of possible parameters when using global or local optimization schemes.

The computational complexity of an algorithm in our framework can be described as the computational complexity of the switch detection algorithm times the number of system states added to the estimated number of parameter switches times the complexity of the integration scheme ($\mathcal{O}(N)$) times the complexity of the optimizer. This can be written with the expression

$$m * \mathcal{O}(f_{SD}(N)) + \hat{N}_s * \mathcal{O}(N * f_{opt}(N, n_p)), \quad (2.4)$$

where $(*)$ represents standard multiplication, f_{SD} is the complexity of the switch detection algorithm and f_{opt} is the complexity of the optimizer which depends on the number of data points and number of parameters. Let us note that since the number of data points is known in the problem statement, the step-size between data points cannot change, thus making the complexity of the integration scheme $\mathcal{O}(N)$. Let us also note that in the most computationally complex scenarios that still achieve an optimal solution (convergence), we would have $\hat{N}_s \approx N$, $f_{SD}(N) = N^2$, $m \geq N$. From here we can see that the rightmost term typically dominates in most use cases. In the examples, we present the overall complexity is $\mathcal{O}(N^2)$ which is granted by the dominance of the rightmost term. This also is the minimum complexity that can be achieved by algorithms within our framework.

2.4. Sparse least absolute shrinkage and selection operator (LASSO) regression for continuously varying parameters

Sparse regression and dictionary methods have been successfully used across several domains to accurately fit parametric curves to signals in the context of dynamical systems [54]. Sparse signal reconstruction on fixed and adaptive supervised dictionary learning for transient stability assessment is provided

by Dabou *et al.* [55]. Devito *et al.* provide a dictionary optimization method for reconstruction of compressed electrocardiogram signals [56]. A framework for signal processing using dictionaries, atoms and deep learning is given by Zhang & Van Der Baan [57]. Given the advances in using dictionaries for sparse regression, we use it for the purposes of identifying the structures of continuously changing parameters in parameter-varying systems.

Suppose that we wish to capture a continuously changing parameter $p(t) : \mathbb{R} \mapsto \mathbb{R}$ as opposed to a piecewise-constant parameter that changes a discrete number of times $p = \{p_1, \dots, p_n\}$, where n is the number of segmented data intervals. We can estimate $p(t)$ using our framework as constructed with some specifications on switch detection and an additional step of sparse regression to identify the continuously changing parameters. To do this, we segment the data into $\frac{N}{6}$ intervals where N is the number of data points in the input signal states. This ensures that the parameters of a given system are not over- or under-sampled. Once our parameters have been sampled, we use the sparse least absolute shrinkage and selection operator (LASSO) regression to fit a function to each parameter where each function is a linear combination of a dictionary of functions. Let us denote this dictionary as $\mathcal{D}_\theta(t)$, where each column of this matrix is a parameterized function denoted as $f_i(\theta_i, t)$, where $i \in \{1, \dots, r\}$ and r is the number of dictionary functions, and each θ_i is a set of parameters for f_i . The linear combinations of these dictionary functions approximate each parameter, $p(t)$.

However, in practice, we cannot assume a well-chosen dictionary, convexity of the objective function, uniqueness of a solution or reasonable dimensionality. While these outlined challenges are largely unresolved in the literature, we can use LASSO to promote solutions $\mathcal{D}_\theta(t)w$ that approximate $p(t)$ with minimal terms. In an applied computational setting, let us denote p as a finite vector approximation of the continuous time-varying parameter $p(t)$. Then our problem statement can be written as

$$\min_{\theta, w} \|p - \mathcal{D}_\theta w\|_2 + \lambda \|w\|_0, \quad (2.5)$$

where $\lambda \in \mathbb{R}$ is the sparse regularization parameter, and $w \in \mathbb{R}^r$ is the vector of weights. The objective function seen in equation (2.5) allows us to achieve an approximation of the parameter $p \approx \mathcal{D}_\theta w$ with only a sparse subset of our dictionary, granted to us by the $\lambda \|w\|_0$ term. Note that that 1-norm for the term $\lambda \|w\|_1$ may grant better results when using gradient-based optimization methods and in other cases.

3. Results and performance

We first analyse our framework's performance on a nonlinear, parameter-varying system which has served as inspiration for the development of our algorithm. The model for the parameter-varying toggle switch (PVTs) is given below and has been analysed in [58]. The parameters in this model have been shown to be globally unidentifiable in [59], so our goal is to achieve accurate parameter estimates in spite of this.

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} \frac{\alpha_1(t)}{1+(y/k_y)^{n_y}} - \delta_1(t)x \\ \frac{\alpha_2(t)}{1+(x/k_x)^{n_x}} - \delta_2(t)y \end{pmatrix}, \quad (3.1)$$

$$\alpha_1 = \alpha_2 = \begin{cases} 1.0 & \text{if } t \in [0, 12], \\ 8.0 & \text{if } t \in (12, 24], \end{cases} \quad (3.2)$$

$$\delta_1 = \delta_2 = \begin{cases} 0.3 & \text{if } t \in [0, 12], \\ 0.6 & \text{if } t \in (12, 24], \end{cases} \quad (3.3)$$

$$k_x = k_y = 1.0, \quad (3.4)$$

$$n_x = n_y = 3.35, \quad (3.5)$$

where x and y represent the concentrations of repressor one and repressor two, respectively; $\alpha_1(t)$ and $\alpha_2(t)$ denote the effective rates of synthesis of repressor one and two, respectively; $\delta_1(t)$ and $\delta_2(t)$ are the self decay rates of repressor one, repressor two, respectively; n_x and n_y represent the respective cooperativities of repression for promoter one and promoter two, respectively; and the Michaelis constants k_x and k_y are respective binding affinities.

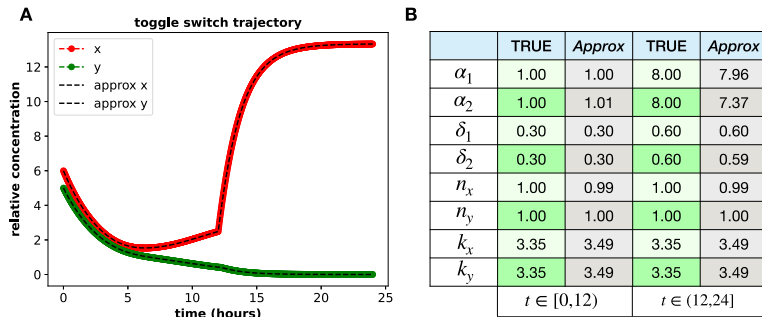


Figure 3. (A) Solution to parameter varying toggle switch and reconstruction of solution with estimated parameters. (B) Estimated parameters for parameter varying toggle switch.

We simulated $N = 1000$ data points from this model using the Livermore solver for ODEs, automatic (LSODA) as the ODE solver. The algorithm substeps we used were: `binseg` for switch detection with auto-regressive cost, $\sigma = 10^{-5}$, $s_g = 5$, differential evolution for a global parameter search where each parameter is in $[0, 10.00]$ over a given time interval and forward Euler for numerical integration of the model during the optimization step. Our switch detection method achieved accuracy in terms of $H_s = 0$ and $E_{N_s} = 0$. When assuming $n_{x,y}$ and $k_{x,y}$ to be static parameters as described in §2.3, we have $E_p = 0.08$ and $E_t = 0.0041$ for the given PVTs as opposed to leaving this assumption out of our algorithm for a worse performance of $E_p = 0.2$ and $E_t = 0.0041$. Thus, instantiating a subset of parameters as constant granted us more accurate parameter estimates than if we had assumed all parameters to be varying. A summary of these results is provided in figure 3.

We see that we are able to accurately estimate the static and bi-phasic parameters of the PVTs as desired. Achieving this in our framework was non-trivial in that the parameter space contains several local optima over each time interval. To obtain the level of accuracy we have shown, we employed the global optimization scheme, differential evolution, to minimize E_t , and we made the *a priori* instantiation of the n_{xy} and k_{xy} parameters to be estimated as constant functions. Using Nelder–Mead or Powell’s methods as opposed to the differential evolution gave us worse errors of $E_t = 0.0049$ and $E_p = 0.79$.

3.1. PDEs and parametric curves

To illustrate how our framework can handle PDEs, we set up the pertinent algorithmic substeps and we ran the algorithm on the heat equation where the diffusion coefficient is the parameter varying function $D(t) = p_1(t)t^{p_2(t)}$, as shown in equation (3.6), where $p_1(t)$ and $p_2(t)$ are piecewise functions in time. A physical understanding of this phenomenon is provided in appendix A.

$$\frac{\partial}{\partial t} u(t, x) = p_1(t)t^{p_2(t)} \frac{\partial^2}{\partial x^2} u(t, x), \quad (3.6)$$

$$\text{where } p_1(t) = \begin{cases} 0.1 & \text{if } t \in [0, 0.25], \\ 2.1 & \text{if } t \in (0.25, 0.5], \\ 4.1 & \text{if } t \in (0.5, 0.75], \\ 6.1 & \text{if } t \in (0.75, 1], \end{cases} \quad (3.7)$$

$$p_2(t) = \begin{cases} 0.5 & \text{if } t \in [0, 0.25], \\ 0.4 & \text{if } t \in (0.25, 0.5], \\ 0.3 & \text{if } t \in (0.5, 0.75], \\ 0.2 & \text{if } t \in (0.75, 1], \end{cases} \quad (3.8)$$

$$\text{with initial condition } u(t=0, x) = \begin{cases} x & \text{if } x \in [0, \frac{\pi}{2}], \\ \pi - x & \text{if } x \in (\frac{\pi}{2}, \pi], \end{cases} \quad (3.9)$$

$$\text{and with boundary condition } u(t, x=0) = u(t, x=\pi) = 0. \quad (3.10)$$

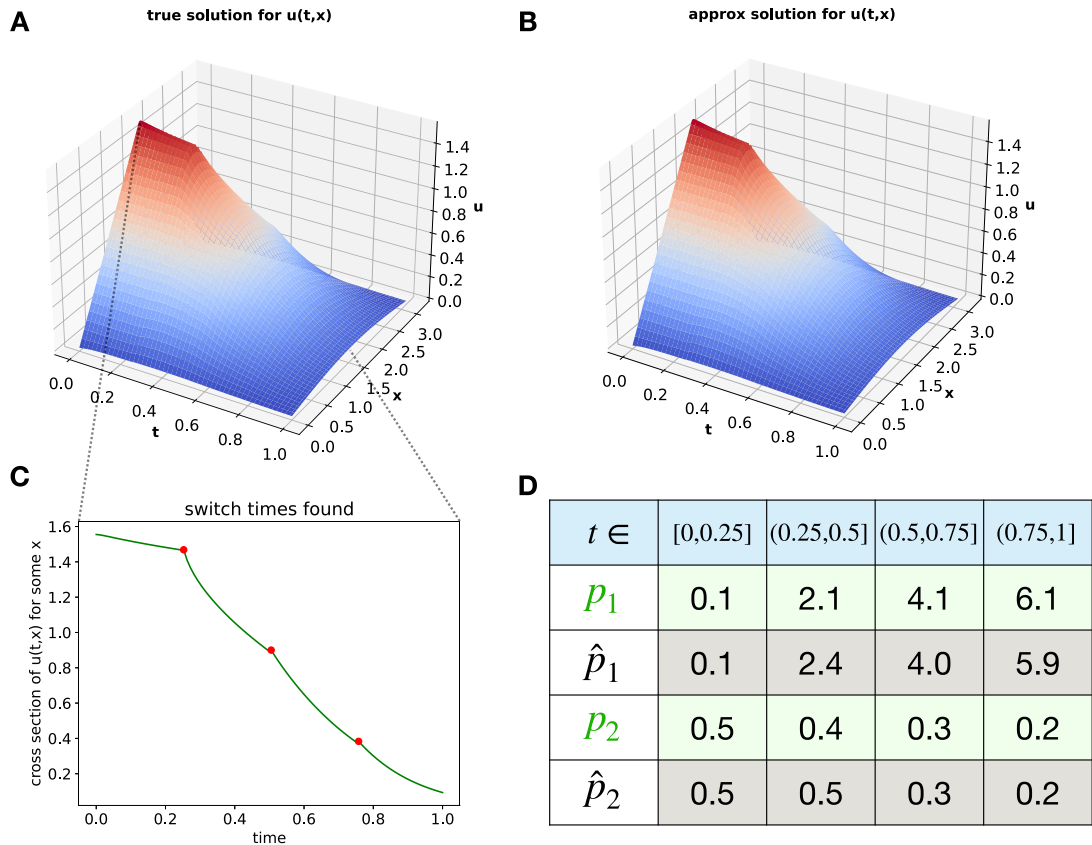


Figure 4. (A) True solution to heat equation PDE. (B) Reconstruction of solution to heat equation PDE with estimated parameters. (C) Switches detected from cross-sectional slice of PDE data. (D) Estimated parameters for heat equation PDE from data.

We used the method of lines (discretization of spatial derivatives) as explicitly written in [equation \(3.6\)](#) to represent the PDE as a system of ODEs with 100 uniformly spaced grid points in x ($M = 100$) and 100 time points over the domain $(t, x) = [0, 1] \times [0, \pi]$. The ODE solver in the optimization substep was forward Euler, we used `binseg` for switch detection with auto-regressive cost, $\sigma = 10^{-3}$, $s_g = 10$ and Nelder–Mead for locally optimal parameters with no pre-set bounds. We see that we have achieved $H_s = 0$, $E_{N_s} = 0$, $E_p = 0.19$ and $E_t = 0.001$ via the algorithm specifications. Thus, we have shown that that an algorithm specified within our framework can accurately estimate the varying parameters of a PDE. A summary of these results is provided in [figure 4](#).

3.2. Robustness to noise and time-series resolution

Quantifying the noise-robustness of our framework is needed given the ever-increasing need to process and understand noisy data for system identification and control [60–62]. To quantify the robustness of our algorithm, we added white noise (zero mean and finite variance) to synthetic data that was generated from three different known ground-truth models which all have distinct orders, numbers of parameters and system structures. We then ran this noisy data through our algorithm and identified relationships between the noisiness of the data and the types of error listed in [table 1](#) which are inherent to our algorithmic substeps.

To illustrate, one of the ground-truth models we used is a biological system given in [equation \(3.11\)](#) where m is mRNA concentration, p is protein concentration, $\alpha_{m,p}$ are respective rates of synthesis and $\delta_{m,p}$ are the respective degradation rates. This is a standard protein synthesis model in which mRNA is transcribed into a given protein. Note that in this model, we have that the protein production is dependent on the transcription of mRNA, but not vice versa. Hence, the model is in agreement with the central dogma of biology.

A motivating factor for this work was to validate novel genetic circuit models with time-varying kinetic rates such as the one below. Based on the equations provided, it is clear that the $\alpha_{m,p}$ rates of

Table 1. Metrics for evaluation of algorithm performance.

error type	description	formula
parameter error	gap between true parameters and estimated parameters	$E_p := \frac{\ p(t_d) - \hat{p}\ _2}{n_p}$
trajectory error	gap between input data and predicted trajectory from model	$E_t := \frac{\ x_{data} - x_{model}\ _2}{N}$
switch number error	difference between true number of switches and those detected	$E_{N_s} := N_s - \hat{N}_s$
switch distance	the worst error among all of the detected switches	$H_s := \text{equation (2.1)}$

synthesis are the varying parameters of the model, and that the respective degradation rates are modelled as unchanging. In an experimental setting, the data that would be collected would be subjected to some measurement noise from the instruments that are used to quantify time-series gene expression levels:

$$\dot{m} = \alpha_m(t) - \delta_m(t)m, \quad (3.11)$$

$$\dot{p} = \alpha_p(t)m - \delta_p(t)p, \quad (3.12)$$

$$\alpha_m(t) = \alpha_p(t) = \begin{cases} 4.0 & \text{if } t \in [0, 12], \\ 5.0 & \text{if } t \in (12, 24], \end{cases} \quad (3.13)$$

$$\delta_m(t) = \delta_p(t) = 1 \quad \forall t \in [0, 24]. \quad (3.14)$$

Let the observables of our synthetic data be denoted as $x := [m \ p]^T$ and let the white noise we add to it be denoted by v_w . Each element of the white noise vector v_w is sampled from a normal distribution with zero mean and finite standard deviation, σ . Our algorithm was used to estimate the time-varying parameters of the given model in [equation \(3.11\)](#) using $x_{data} := x + v_w$. A subset of these runs are found in [figure 5](#), where we manually set the number of switches to be detected in the data to be $N_s = 1$ so that we could measure noise effects in this idealized best-case scenario. In the worst-case scenario where the number of parameter switches, N_s , is unknown, switch detection algorithms often mistake jumps in the data from the noise as switches which leads to worse performance than what we show here. This worst-case scenario often gives relatively accurate data reconstruction, but unreliable parameters and untrustworthy detection of switches.

The ODE solver we used for the numerical integration in the optimization substep was forward Euler and the algorithm substeps we used were: `binseg` for switch detection with auto-regressive cost, $\sigma = N_s = 1$, $s_g = 10$ and Nelder–Mead for locally optimal parameters with no pre-set bounds. We used forward Euler here because LSODA performed worse than Euler's method within computing the objective function. Time integration errors are likely to affect the estimation, since the loss function depends on the rolled-out solution. To reduce such errors, one can utilize the modular nature of our framework to implement other ODE solvers just as we have done.

From both [figures 5](#) and [6](#), it is clear that our method for parameter estimation shows increases in all error metrics with increases to the noise in data. There is a cascading effect of error propagation owing to noise (v_w): `binseg` inaccurately detects switches which leads to inaccurate data segmentation ($\uparrow D_s$ and $\uparrow E_{N_s}$) which causes inaccurate parameter fitting (E_p), and thus, parameterized model trajectories inaccurately approximate the data (E_t). This cascading effect is summarized as

$$\uparrow \|v_w\| \implies (\uparrow H_s \text{ and } \uparrow E_{N_s}) \implies \uparrow E_p \implies \uparrow E_t.$$

This cascading propagation of error is one of the drawbacks of algorithms in our framework. Methodological substeps specializing in the robust handling of noisy data can be added on at various points within our framework for improvements in this weak-point, but this is outside the scope of this paper and may be further expanded on in future work.

To examine how our framework tends to handle low-resolution time-series data, we ran a similar test as before, but without noise to limit the effect of confounding variables in our results. From [figure 7](#), we see that switch detection performs well regardless of the resolution of our data. We also see that the parameter estimates and thus the reconstruction of the data is marginally worse at low resolution.

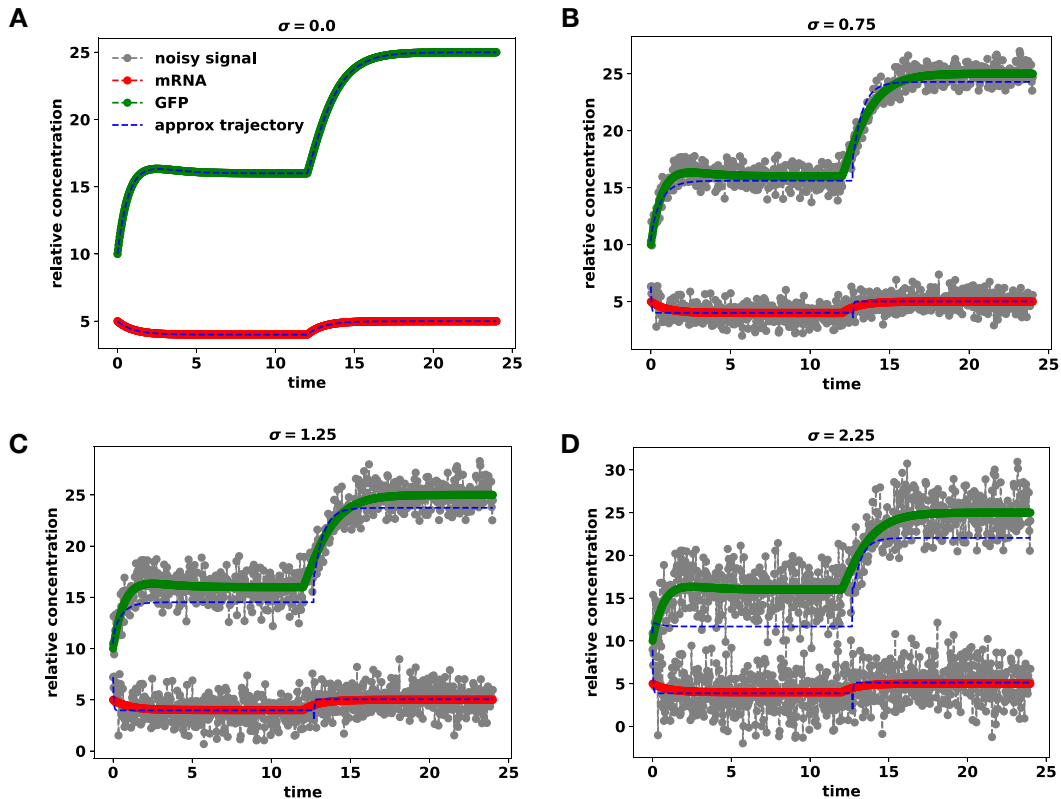


Figure 5. (A) Synthetic data and reconstruction from estimated parameters. (B) Synthetic data with white noise ($\sigma = 0.75$) and reconstruction from estimated parameters. (C) Synthetic data with white noise ($\sigma = 1.25$) and reconstruction from estimated parameters. (D) Synthetic data with white noise ($\sigma = 2.25$) and reconstruction from estimated parameters.

3.3. Switching parameters at non-uniform intervals

The special handling of our adapted switch detection framework to detect discretely switching parameters in dynamical systems is heavily motivated by cases in which parameters across the states of a dynamical system are switching independently from each other. Suppose now that we have the same system as in equation (3.11) but with parameters that switch non-uniformly on different time intervals as depicted in equation (3.16).

$$\dot{m} = \alpha_m(t) - \delta_m(t)m, \quad (3.16)$$

$$\dot{p} = \alpha_p(t)m - \delta_p(t)p, \quad (3.17)$$

$$\text{where } \alpha_m(t) = \begin{cases} 4.0 & \text{if } t \in [0, 5], \\ 5.0 & \text{if } t \in (5, 15], \\ 6.0 & \text{if } t \in (15, 24], \end{cases} \quad (3.18)$$

$$\alpha_p(t) = \begin{cases} 4.0 & \text{if } t \in [0, 5], \\ 5.0 & \text{if } t \in (5, 10], \\ 6.0 & \text{if } t \in (10, 15], \\ 7.0 & \text{if } t \in (15, 20], \\ 8.0 & \text{if } t \in (20, 25], \end{cases} \quad (3.19)$$

$$d_m(t) = d_p(t) = \begin{cases} 1 & \text{if } t \in [0, 25]. \end{cases} \quad (3.20)$$

We test our algorithm on the above system to show that our adaptation of `binseg` is able to handle these scenarios as shown in figure 8. The ODE solver we used for the numerical integration in the optimization substep was forward Euler and the algorithm substeps we used were: `binseg` for switch detection with

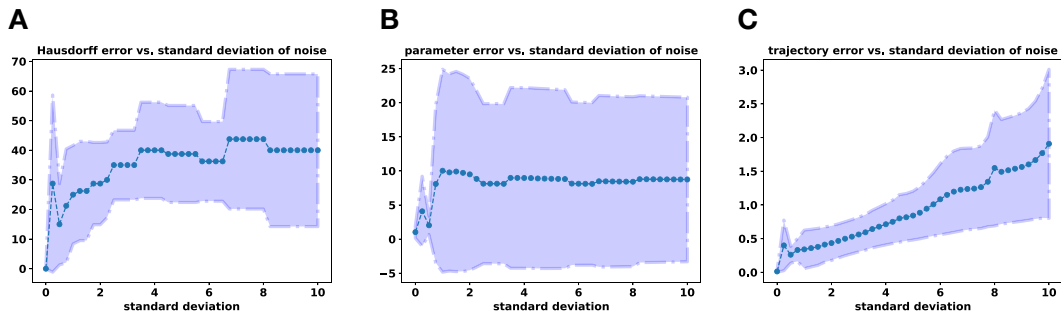


Figure 6. (A) Hausdorff error versus standard deviation of noise. (B) Parameter error versus standard deviation of noise. (C) Trajectory error versus standard deviation of noise. Dotted lines are the averages over several runs and dynamical systems. Shaded regions are standard deviation error bars computed from all runs.

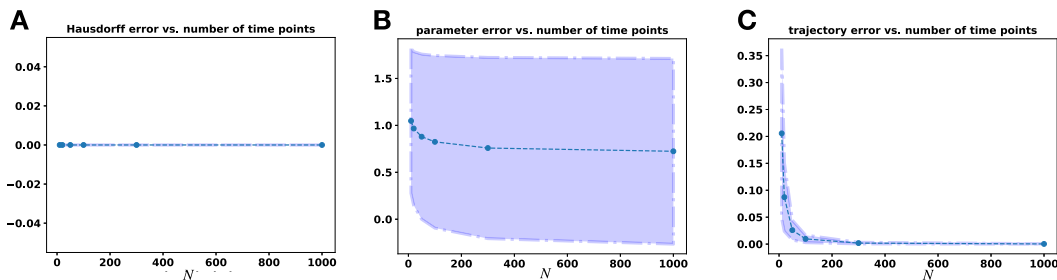


Figure 7. (A) Hausdorff error versus number of time-series data points. (B) Parameter error versus number of time-series data points. (C) Trajectory error versus number of time-series data points. Dotted lines are the averages over several runs and dynamical systems. Shaded regions are standard deviation error bars computed from all runs.

auto-regressive cost, $\sigma = N_s = 1$, $s_g = 10$ and Nelder–Mead for locally optimal parameters with no pre-set bounds. With this algorithmic set-up, we achieved $H_s = 0$, $E_{N_s} = 0$, $E_t = 0.001$ and $E_p = 0.05$.

3.4. Sample method comparison

While accurately detecting a system’s changing parameters has been achieved for certain classes of systems (most notably with methods developed by Yong *et al.* and Curi *et al.*, respectively [63,64]), our paper presents a generalized framework for estimating the switching parameters of systems, their switch locations, and for continuously varying parameters. Although several key differences exist between our framework and existing methods, we now present a comparison using standard methods and metrics found in the literature. To do this comparison fairly, it is important to note that we used the methodology from [64] to directly compare the Adal-IE method’s stated results, the UMV-le method’s results [65] shown in table 1 of [64], and the results of our chosen method.

The experimental methodology involves simulating modelling mismatch in our system taken from Khalil’s example 4.27 [66]. To simulate this modelling mismatch, we perturb the coefficients of the state variables in the given vector field as shown in equation (3.21):

$$\dot{x} = -(1 + \eta_1)x + (1 + \eta_2)y^2, \quad (3.21)$$

$$\dot{y} = -(1 + \eta_3)y + u(t), \quad (3.22)$$

$$\text{where } \eta_{1,2,3} \sim \mathcal{N}(0, 1). \quad (3.23)$$

We ran 20 episodes of input estimation for both inputs that we tested. Each simulation consisted of 100 time steps. An RMSE value was calculated for each episode and we have presented the mean RMSE value over these 20 episodes per input in table 2. In the experiment of Curi *et al.* [64] noise was injected into the entries of the state transition matrix of the linearized and discretized system (as necessitated since Adal-IE and UMV-IE only work for linear discrete-time systems), which inherently incurs some linearization error that our method does not since it can deal directly with nonlinear systems. This is the reality of several of the methods that exist in the literature, and it showcases an advantage of our framework.

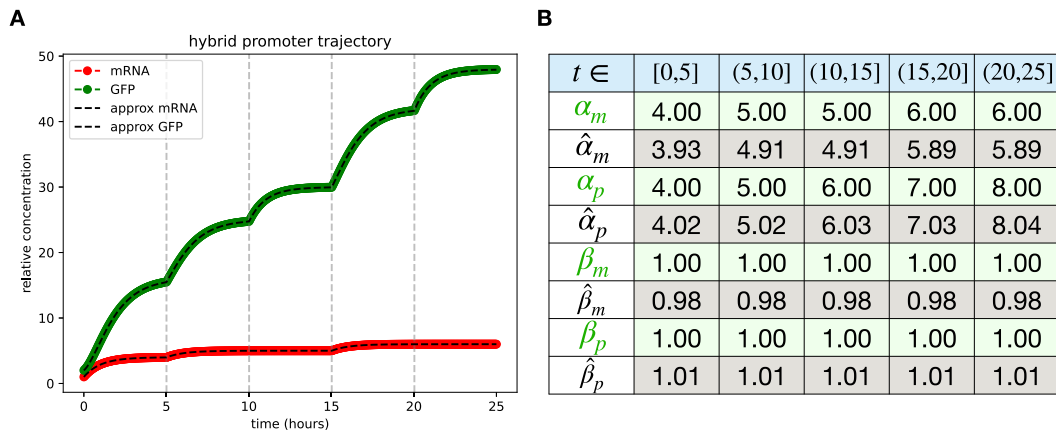


Figure 8. (A) Trajectory of hybrid promoter model with different parameters ($\alpha_m(t)$ and $\alpha_p(t)$) switching at different time intervals. Switch locations are shown as grey dotted lines (estimated = true). (B) Approximated parameters of hybrid promoter model in which parameters switch at different time intervals.

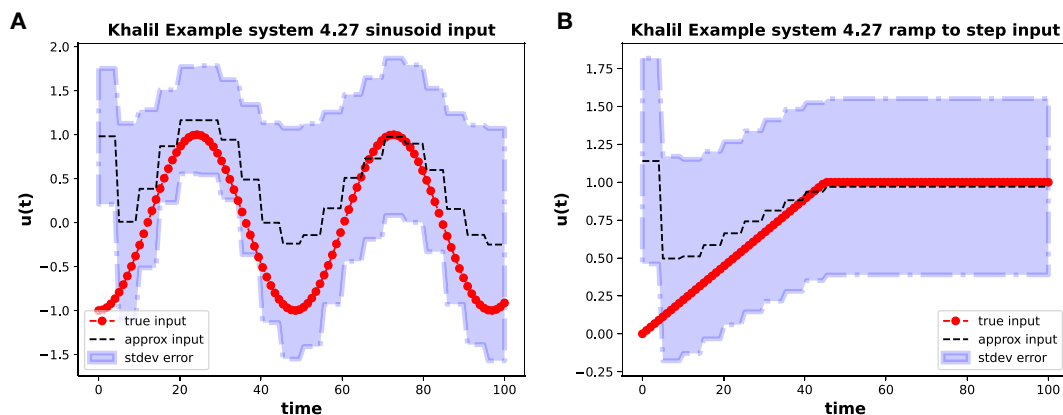


Figure 9. (A) True sinusoidal input (red) plotted with mean of input estimates (black) with standard deviation error bars (blue). (B) True ramp to step input (red) plotted with mean of input estimates (black) with standard deviation error bars (blue).

To summarize the results of our comparison, let us start by noting our method's trade-off of being able to obtain the smallest mean RMSE values listed in table 2, while also obtaining greater variance on the estimated varying input/parameter seen in figure 9. To achieve these results with our framework, we imposed a bound of $|\hat{u}(t)| \leq 2$ during the optimization step of our algorithm. The larger the bound, the worse our parameter estimates were, but we were still able to achieve superior mean RMSE values with bounds as large as $|\hat{u}(t)| \leq 10$. Without implementing bounds at all, there was a chance for our method to obtain worse performance across the board.

3.5. Sparse dictionary regression for continuously varying parameters

To test our method outlined in §2.4, we estimate the time-varying parameter $\alpha(t)$ in the PDE model for advection–diffusion found in equation (3.24) using an algorithm specified within our framework adjoined with sparse regression. The main differences between this example and that of §3.1 are that we now have an advection term in the PDE denoted by the first-order spatial derivative and that we are not assuming knowledge of the structure of our varying parameter, $\alpha(t)$. We proceed in the estimation of $\alpha(t)$ as a continuously varying parameter for two reasons: owing to the horrendous accuracy obtained if estimating all parameters as constant, and the lack of coherent discrete switches detected in the data. Intuitively, it is clear from the data found in figure 10 that the direction and speed of advection are changing. We assumed in our test run that the diffusion coefficient D was to be estimated as a constant parameter.

Table 2. Table of mean RMSE values obtained from computational experiments in which the following inputs were estimated (A) sinusoidal (B) ramp-to-step.

	NonLin 4.27	method
A)	0.190	ours
	0.500	Adal-IE
	1.678	UMV-IE
(B)	0.113	ours
	0.261	Adal-IE
	1.681	UMV-IE

$$\frac{\partial}{\partial t} u(t, x) = \alpha(t) \frac{\partial}{\partial x} u(t, x) + D \frac{\partial^2}{\partial x^2} u(t, x), \quad (3.24)$$

$$\text{where } \alpha(t) = \sin(3t - 1) \quad \text{and} \quad D = 0.01, \quad (3.25)$$

$$\text{with initial condition } u(0, x) = \begin{cases} \sin(3x) & \text{if } x \in [0, \pi], \\ 0 & \text{if else.} \end{cases} \quad (3.26)$$

We used the method of lines to set up this PDE and we used LSODA to simulate the data with 100 uniformly spaced grid points in x ($M = 100$) and 100 time points over the domain $(t, x) = [0, 1] \times [-\pi/4, \pi]$. The ODE solver we used for the numerical integration in the optimization substep was forward Euler and the algorithm substeps we used were: `binseg` for data segmentation with auto-regressive cost, the number of assumed switches as $\sigma = N_s = N/6$, the switch gap as $s_g = 1$, Nelder–Mead for locally optimal parameters with pre-set bounds such that $|\alpha(t)| < 10$ and $|D| < 10$, and sparse regression with regularization term as $\lambda \|w\|_1$, where $\lambda = 0.01$. With this algorithmic set-up, we achieved $E_t = 0.001$ and $E_p = 0.001$.

Without our implementation of sparse regression, we achieved a parameter error of $E_p = 0.16$. With sparse regression we achieved a parameter error of $E_p = 0.05$, thus showcasing an improvement in parameter estimation accuracy through the utilization of sparse regression. Further, sparse regression has allowed us to reconstruct the parameterization of $\alpha(t)$ as $\hat{\alpha}(t) = 0.97 \sin(3t + 11.5) + \epsilon$ where ϵ contains the minuscule terms from the other dictionary functions, namely, $\epsilon = (10^{-13})e^{-2.55t^{0.6}} + 0.01t^{0.96}$. Note that the offset of -1 radians in $\alpha(t)$ is approximately equal to the 11.5 radian offset in $\hat{\alpha}(t)$.

3.6. Parameter estimation on time-varying gene-circuit data

We have designed a genetic circuit corresponding to [equation \(3.11\)](#) and have captured pertinent gene expression data to show the practical applicability of our framework. Our data collection ran for 48 h measuring both a green fluorescent protein (GFP) expression and optical density (OD) every 3 min, giving us 960 data points. We were only able to measure GFP over time and not able to measure mRNA, so we proceed by amalgamating two parameters to get the $\theta(t) = \alpha_p(t)m(t)$ parameter, giving rise to the model

$$\dot{p} = \theta(t) - \delta_p(t)p. \quad (3.26)$$

We use `binseg` to segment our gene expression data. Next, we impose the bounds $|\delta_p(t)| \leq \epsilon_\delta$ over all segmented time intervals with the conservative assumption that $\epsilon_\delta \in (0, 0.08]$, based on the degradation rates of the protein (sfGFP) found in the literature [67,68]. We say that our ϵ_δ interval is conservative because it includes unlikely degradation rates closer to the extreme ranges of our assumed interval, thus giving us more uncertainty in our estimates of $\theta(t)$. With this assumption, we fit the model to the gene expression data three times: once with the constant minimum value of $\delta_p = 10^{-5} \approx 0$, once with the constant maximum value of $\delta_p = 0.08$ and once where both $\theta(t)$ and $\delta_p(t)$ are inferred simultaneously. Applying our framework in this way with our reduced-order model allows us to get parameter error bounds in accordance with parameter identifiability theory [7,11,59,69].

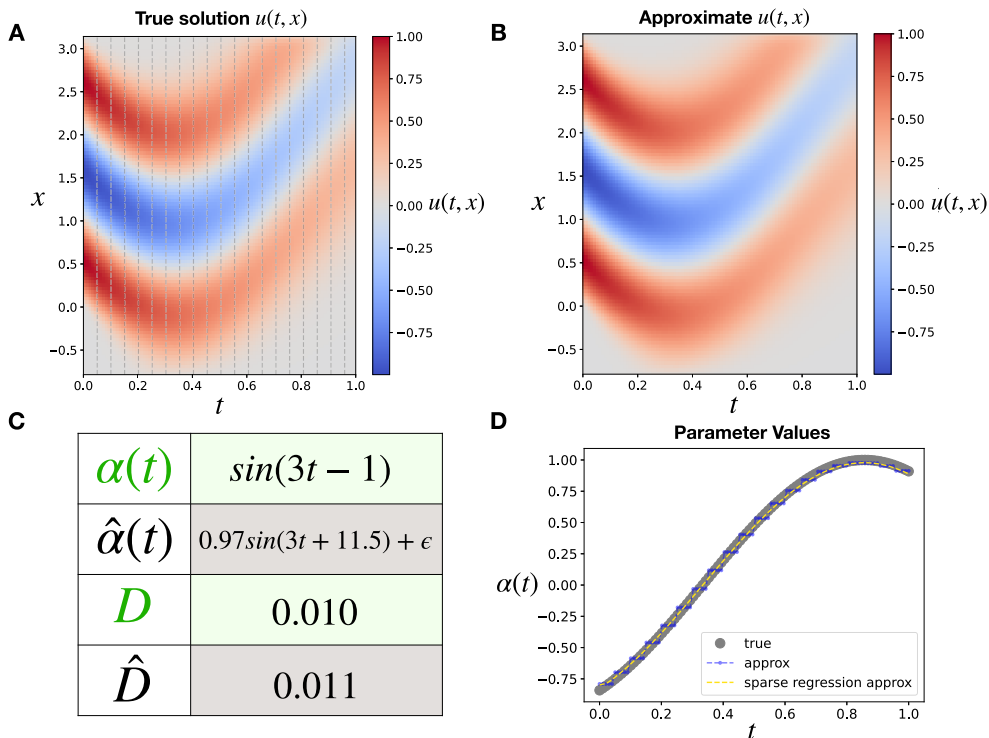


Figure 10. (A) Synthetic data from advection–diffusion equation containing a continuously varying sinusoidal parameter. The vertical grey dotted lines indicate the segmentation of the data for the sampling of the continuously varying parameter. (B) Reconstruction of heat equation data using estimated model parameters from algorithm with sparse regression. (C) Table of parametrized true and estimated parameters from the set of dictionary functions. (D) True and approximate parameters from our algorithm with and without sparse regression.

The ODE solver we used for the numerical integration in the optimization substep was forward Euler and the algorithm substeps we used were: binseg for data segmentation with auto-regressive cost, the noise estimate used was $\sigma = 0.1$, the switch gap as $s_g = 1$ and Powell’s method for locally optimal parameters with pre-set bounds such that $|\theta(t)| < \max_t(p(t))$ and $|\delta_p| < 0.08$. With the algorithmic set-up described in the previous section, we achieved $E_t = 0.001$.

We use this optimization-based algorithm to fit the parameters in this way for two main reasons. The first reason is that it is one of the only algorithms with demonstrable accuracy which has been designed and suited for this explicit purpose. The second reason is that if we wanted to use an alternate parameter estimation technique such as Bayesian inferencing [70], then we would need a distribution of parameter estimates that was gathered from our data in the first place. So in this scenario, our algorithm would serve as a promising first step in statistical approaches to parameter estimation.

We illustrate the results of our parametrized hybrid promoter library via sample taken from the library in figure 11. Here, we can see that our algorithm is able to detect switches in our gene expression data as well as reconstruct the data accurately upon estimation of the corresponding model parameters.

4. Discussion and conclusions

With our framework, systems which contain only discrete parameter switches can have their parameters and switch times accurately estimated. Further, scenarios which have only continuously varying parameters are able to be estimated accurately with the use sparse regression on the sampled parameter estimates as shown. Working with parameters that are combinations of both discretely switching and continuously varying is successful up to the following extent: we can approximate these parameters/inputs reasonably well by sufficiently segmenting the data, but we cannot estimate the switch locations during this process. In the broader context of the literature, our method is the first to attempt to estimate the switch locations of parameters while also estimating the parameters themselves. Upon reading our results, one might have thought to try running the dynamic programming switch detection algorithm

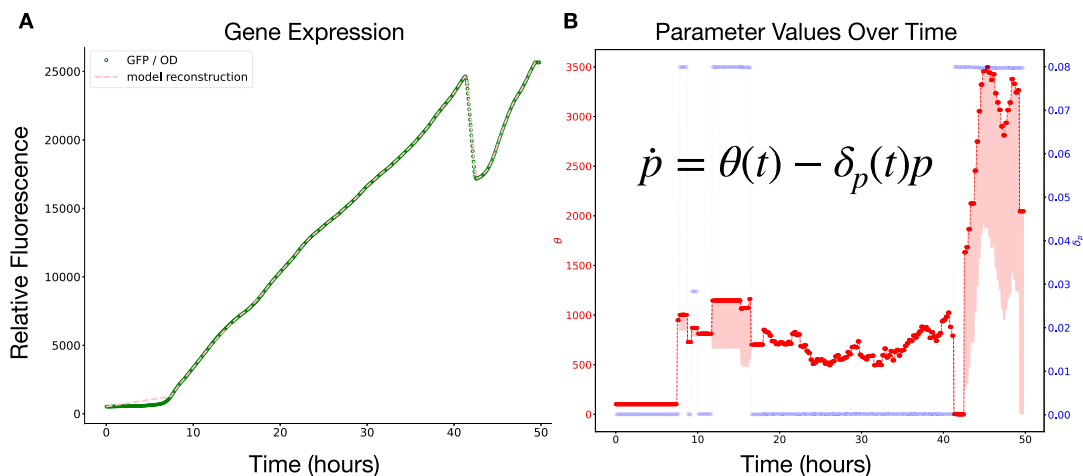


Figure 11. (A) Reconstructed gene expression profile for a given hybrid promoter cassette. (B) Parameter value estimates with standard deviation error bars.

where N_s is increased every iteration until the trajectory error is below some threshold. While this idea is somewhat sound upon reading, the method suffers greatly in practice with real data owing to the following: (i) If N_s is unknown then the user must have some insight into what error threshold to pick to give a reasonable estimate of N_s ; (ii) this method is extremely computationally costly and often gives unreasonably high N_s estimates with real, noisy data; (iii) in general, one is better off adjusting the hyper parameters of a switch detection algorithm like `binseg` in a practical setting with real, noisy data.

In conclusion, we present a modular framework for estimating varying parameters in dynamical systems, focusing initially on the sub-class of problems where parameters switch discretely and are modelled as piecewise constant functions. By combining the strengths of switch detection, numerical integration and optimization into a cohesive structure, our framework offers accuracy and flexibility to accommodate user-specific requirements. Our approach has been validated across diverse examples, including biological systems and physical models, demonstrating its adaptability to scenarios with fixed and switching parameters, non-uniform switching and time-varying dynamics. Moreover, we characterized the framework's robustness to measurement noise and extended its capabilities to continuously varying parameters using dictionary-based sparse regression with trigonometric and polynomial functions. This work establishes a foundation for addressing a wide range of parameter-varying system problems and provides a versatile tool set for advancing research in dynamical system modelling and control.

Ethics. This work did not require ethical approval from a human subject or animal welfare committee.

Data accessibility. All code used to generate the figures in this paper can be found at [71].

Declaration of AI use. We have not used AI-assisted technologies in creating this article.

Authors' contributions. J.M.H.: conceptualization, data curation, formal analysis, investigation, methodology, software, validation, visualization, writing—original draft, writing—review and editing; E.Y.: formal analysis, investigation, methodology, project administration, resources, supervision, writing—review and editing.

Both authors gave final approval for publication and agreed to be held accountable for the work performed therein.

Conflict of interest declaration. We declare we have no competing interests.

Funding. This work was funded in part by an NSF CAREER Award 2240176, the Army Young Investigator Program Award W911NF2010165 and the Institute of Collaborative Biotechnologies/Army Research Office grants W911NF19D0001, W911NF22F0005, W911NF190026 and W911NF2320006. This work was also supported in part by a subcontract awarded by the Pacific Northwest National Laboratory for the Secure Biosystems Design Science Focus Area 'Persistence Control of Engineered Functions in Complex Soil Microbiomes' sponsored by the U.S. Department of Energy Office of Biological and Environmental Research.

Acknowledgements. For insightful discussions related to this work, the authors would like to thank Charles Johnson, Yanran Wang, Taishi Kotsuka, Harnoor Lali, SIAM and João Hespanha.

Appendix A.

The theoretical physics underlying this example problem provides valuable insight into the behaviour of complex materials under dynamic thermal conditions. In this case, the one-dimensional heat equation

models the temperature distribution along a thin rod with an initial heat profile $u(0, x)$ as defined in equation (3.9). As time progresses, the parameter representing the rod's heat diffusivity, $D(t)$, evolves dynamically. This behaviour reflects the rod's composition—a non-homogeneous composite material consisting of three distinct compounds, each with unique thermal properties and phase-change characteristics.

As the rod cools, the heat diffusivity of one compound undergoes a significant change at its critical temperature, corresponding to the first parameter switch. Similar transitions occur for the other two compounds, resulting in the discrete parameter switches observed in the measured data. Assuming that previous studies have shown that the heat diffusivity of such composites changes exponentially with cooling, this behaviour can be represented by the parameter structure $D(t) = p_1(t)t^{p_2(t)}$. The non-homogeneous nature of the composite material justifies the discrete switching observed in the heat diffusivity.

References

- Ryan EP, Ichikawa A. 1979 Parameter estimation and control for distributed systems. In *IFAC Proceedings Volumes*, vol. **12**, pp. 149–155, Zurich, Switzerland: IFAC Symposium on computer Aided Design of Control Systems. (doi:10.1016/S1474-6670(17)65590-3)
- John AB, Cliff EM, Farlow K. 2014 Parameter estimation and model discrepancy in control systems with delays. In *IFAC Proceedings Volumes*, vol. **47**, pp. 11679–11684, 19th IFAC World Congress. (doi:10.3182/20140824-6-ZA-1003.02294)
- Mitra ED, Hlavacek WS. 2019 Parameter estimation and uncertainty quantification for systems biology models. *Curr. Opin. Syst. Biol.* **18**, 9–18. (doi:10.1016/j.coisb.2019.10.006)
- Penas DR, González P, Egea JA, Doallo R, Banga JR. 2017 Parameter estimation in large-scale systems biology models: a parallel and self-adaptive cooperative strategy. *BMC Bioinform* **18**, 52. (doi:10.1186/s12859-016-1452-4)
- Zhan C, Yeung LF. 2011 Parameter estimation in systems biology models using spline approximation. *BMC Syst. Biol.* **5**, 14. (doi:10.1186/1752-0509-5-14)
- Valderrama-Bahamóndez GI, Fröhlich H. 2019 MCMC techniques for parameter estimation of ode based models in systems biology. *Front. Appl. Math. Stat* **5**, 55. (doi:10.3389/fams.2019.00055)
- Giampiccolo S, Reali F, Fochesato A, Iacca G, Marchetti L. 2024 Robust parameter estimation and identifiability analysis with hybrid neural ordinary differential equations in computational biology. *NPJ Syst. Biol. Appl.* **10**, 139. (doi:10.1038/s41540-024-00460-3)
- Ashyraliyev M, Fomekong-Nanfack Y, Kaandorp JA, Blom JG. 2009 Systems biology: parameter estimation for biochemical models. *FEBS J.* **276**, 886–902. (doi:10.1111/j.1742-4658.2008.06844.x)
- Douspis M, Bartlett JG, Blanchard A, Le Dour M. 2001 Concerning parameter estimation using the cosmic microwave background. *A&A* **368**, 1–14. (doi:10.1051/0004-6361:20000573)
- Bailer-Jones CAL. 2005 Astronomical object classification and parameter estimation with the gaia galactic survey satellite. In *Studies in classification, data analysis, and knowledge organization classification — the ubiquitous challenge*, pp. 325–329. Berlin/Heidelberg: Springer-Verlag. (doi:10.1007/3-540-28084-7_36)
- Martinelli A. 2024 Identifiability of nonlinear ode models with time-varying parameters: the general analytical solution and applications in viral dynamics. *arXiv* (doi:10.2139/ssrn.4340774)
- Wang Z, An H, Luo X. 2019 Switch detection and robust parameter estimation for slowly switched Hammerstein systems. *Nonlinear Anal.: Hybrid Syst.* **32**, 202–213. (doi:10.1016/j.nahs.2018.12.002)
- Zheng W, Wan Y, Yang F, Shang C, Ye H, Jiang M, Wang J. 2024 Identification of switched dynamic system for electric multiple unit train modeling. *Control Eng. Pract.* **143**, 105815. (doi:10.1016/j.conengprac.2023.105815)
- Yu M, Bianchi F, Piroddi L. 2023 A randomized method for the identification of switched NARX systems. *Nonlinear Anal.: Hybrid Syst.* **49**, 101364. (doi:10.1016/j.nahs.2023.101364)
- Bianchi F, Breschi V, Piga D, Piroddi L. 2021 Model structure selection for switched NARX system identification: A randomized approach. *Automatica* **125**, 109415. (doi:10.1016/j.automatica.2020.109415)
- Li H, Zhang K, Tan M. 2022 A novel system identification algorithm for nonlinear Markov jump system. *Inf. Sci.* **616**, 348–366. (doi:10.1016/j.ins.2022.10.102)
- Boddupalli N, Matchen T, Moehlis J. 2023 Symbolic regression via neural networks. *Chaos* **33**, 8. (doi:10.1063/5.0134464)
- Johnson CA, Yeung E. 2024 Nonlinear data-driven control via koopman models: the interplay of stabilization and subspace closure error
- Susuki Y, Eto K, Hiramatsu N, Ishigame A. 2022 Control of Oscillatory Temperature Field in a Building via Damping Assignment to Nonlinear Koopman Mode. In *2022 IEEE Conference on Control Technology and Applications (CCTA)*, Trieste, Italy. 3 Park Avenue, New York, NY: IEEE. (doi:10.1109/CCTA49430.2022.9966014)
- Susuki Y, Eto K, Hiramatsu N, Ishigame A. 2024 Control of in-room temperature field via damping assignment to nonlinear koopman mode. In *2022 IEEE Conference on Control Technology and Applications (CCTA)*, vol. **32**, pp. 1569–1578. (doi:10.1109/TCST.2023.3345149)
- Kon J, van de Wijdeven J, Bruijnen D, Tóth R, Heertjes M, Oomen T. 2023 Direct learning for parameter-varying feedforward control: a neural-network approach. In *2023 62nd IEEE Conference on Decision and Control (CDC)*, Singapore, pp. 3720–3725. (doi:10.1109/CDC49753.2023.

- 10383877)
22. Xu N, Ding F. 2023 Parameter estimation for a class of time-varying systems with the invariant matrix. *Int. J. Robust Nonlinear Control* **33**, 2163–2181. (doi:10.1002/rnc.6488)
23. Cláudio Andrade Souza L, Martinez Palhares R. 2011 Parameter estimation on linear time-varying systems. *J. Franklin Inst.* **348**, 777–789. (doi:10.1016/j.jfranklin.2011.02.007)
24. Johnson RS, Di Cairano S, Sanfelice RG. 2021 Parameter Estimation for Hybrid Dynamical Systems using Hybrid Gradient Descent. In *2021 60th IEEE Conference on Decision and Control (CDC)*, Austin, TX, USA. 3 Park Avenue, New York, NY: IEEE. (doi:10.1109/CDC45484.2021.9682794)
25. Qian L. 2025 Bayesian parameter estimation for linear parameter-varying systems with outliers and missing observations. *IEEE Access* **13**, 215715–215723. (doi:10.1109/ACCESS.2024.3418575)
26. Al-Taie F, Hanan IK, Fadhel FS. 2024 Extended robust controller design for parameter varying multi-dimensional systems. *AIP Conf. Proc.* **3079**, 050005.
27. Sarmin EN, Chudov LA. 1963 On the stability of the numerical integration of systems of ordinary differential equations arising in the use of the straight line method. *USSR Comput. Math. Math. Phys.* **3**, 1537–1543. (doi:10.1016/0041-5553(63)90256-8)
28. Truong C, Oudre L, Vayatis N. 2020 Selective review of offline change point detection methods. *Signal Proc.* **167**, 107299. (doi:10.1016/j.sigpro.2019.107299)
29. Lucas de Oliveira Prates. 2021 A more efficient algorithm to compute the rand index for change-point problems. *arXiv* (doi:10.48550/arXiv.2112.03738)
30. MacKay DJC, Ryan Prescott Adams and David. 2007 Bayesian online changepoint detection. *arXiv* (doi:10.48550/arXiv.0710.3742)
31. Fearnhead P, Killick R, Eckley IA. 2012 Optimal detection of changepoints with a linear computational cost. *J. Am. Stat. Assoc.* **107**, 1590–1598. (doi:10.1080/01621459.2012.737745)
32. Rudin LI, Osher S, Fatemi E. 1992 Nonlinear total variation based noise removal algorithms. *Phys. D: Nonlinear Phenomena* **60**, 1. (doi:10.1016/0167-2789(92)90242-f)
33. Page ES. 1954 Continuous inspection schemes. *Biometrika* **41**, 100. (doi:10.2307/2333009)
34. Jackson B *et al.* 2005 An algorithm for optimal partitioning of data on an interval. *IEEE Signal Process. Lett.* **12**, 105–108. (doi:10.1109/lsp.2001.838216)
35. Rabiner LR. 1989 A tutorial on hidden Markov models and selected applications in speech recognition. *Proc. IEEE* **77**, 257–286. (doi:10.1109/5.18626)
36. Harchaoui Z, Cappe O. 2007 Retrospective multiple change-point estimation with kernels. In *2007 IEEE/SP 14th Workshop on Statistical Signal Processing*. (doi:10.1109/ssp.2007.4301363)
37. Vert JP, Bleakley K. 2010 Fast detection of multiple change-points shared by many signals using group lars (eds J Lafferty, C Williams, J Shawe-Taylor, R Zemel, A Culotta). In *Advances in neural information processing systems*, Vancouver, British Columbia, Canada. vol. 23. Curran Associates, Inc.
38. Liu S, Yamada M, Collier N, Sugiyama M. 2013 Change-point detection in time-series data by relative density-ratio estimation. *Neural Netw.* **43**, 72–83. (doi:10.1016/j.neunet.2013.01.012)
39. Lavielle M, Teyssière G. 2006 Detection of multiple change-points in multivariate time series. *Lith. Math. J.* **46**, 287–306. (doi:10.1007/s10986-006-0028-9)
40. Davis TCMRL, Rodriguez-Yam GA. 2006 Structural break estimation for nonstationary time series models. *J. Am. Stat. Assoc.* **101**, 223–239. (doi:10.1198/016214505000000745)
41. Bai J. 1997 Estimating multiple breaks one at a time. *Econ. Theory* **13**, 315–352. (doi:10.1017/S0266466600005831)
42. Fryzlewicz P. 2014 Wild binary segmentation for multiple change-point detection. *Ann. Statist.* **42**, 2243–2281. (doi:10.1214/14-AOS1245)
43. Savitzky A, Golay MJ. 1964 Smoothing and differentiation of data by simplified least squares procedures. *Anal. Chem.* **36**, 1627–1639.
44. Schmid M, Rath D, Diebold U. 2022 Why and how savitzky-golay filters should be replaced. *ACS Meas. Sci. Au* **2**, 185–196. (doi:10.1021/acsmasuresciau.1c00054)
45. Whittaker ET. 1922 On a new method of graduation. *Proc. Edinb. Math. Soc.* **41**, 63–75. (doi:10.1017/S0013091500077853)
46. Nelder JA, Mead R. 1965 A simplex method for function minimization. *Comput. J.* **7**, 308–313. (doi:10.1093/comjnl/7.4.308)
47. Powell MJD. 1964 An efficient method for finding the minimum of a function of several variables without calculating derivatives. *Comput. J.* **7**, 155–162. (doi:10.1093/comjnl/7.2.155)
48. Gábor A, Banga JR. 2015 Robust and efficient parameter estimation in dynamic models of biological systems. *BMC Syst. Biol.* **9**, 74. (doi:10.1186/s12918-015-0219-2)
49. Wales DJ, Doye JPK. 1997 Global optimization by basin-hopping and the lowest energy structures of lennard-jones clusters containing up to 110 atoms. *J. Phys. Chem.* **101**, 5111–5116. (doi:10.1021/jp970984n)
50. Lampinen J, Storn R. 2004 *Differential evolution*, pp. 123–166. Berlin Heidelberg, Berlin, Heidelberg: Springer. (doi:10.1007/978-3-540-39930-8_6)
51. Righini G. 1995 A double annealing algorithm for discrete location/allocation problems. *Eur. J. Oper. Res.* **86**, 452–468.
52. Endres SC, Sandrock C, Focke WW. 2018 A simplicial homology algorithm for Lipschitz optimisation. *J. Glob. Optim.* **72**, 181–217. (doi:10.1007/s10898-018-0645-y)
53. Jones DR, Perttunen CD, Stuckman BE. 1993 Lipschitzian optimization without the Lipschitz constant. *J. Optim. Theory Appl.* **79**, 157–181. (doi:10.1007/BF00941892)

54. Brunton SL, Proctor JL, Kutz JN. 2016 Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proc. Natl Acad. Sci. USA* **113**, 3932–3937. (doi:10.1073/pnas.1517384113)
55. Dabou RT, Kamwa I, Tagoudjeu J, Mugombozi FC. 2021 Sparse signal reconstruction on fixed and adaptive supervised dictionary learning for transient stability assessment. *Energies* **14**, 7995. (doi:10.3390/en14237995)
56. De Vito L, Picariello E, Picariello F, Rapuano S, Tudosa I. 2021 A dictionary optimization method for reconstruction of ECG signals after compressed sensing. *Sensors* **21**, 5282. (doi:10.3390/s21165282)
57. Zhang C, Van Der Baan M. 2022 Signal processing using dictionaries, atoms, and deep learning: a common analysis-synthesis framework. *Proc. IEEE* **110**, 454–475. (doi:10.1109/JPROC.2022.3155904)
58. Harrison J, Yeung E. 2021 Stability analysis of parameter varying genetic toggle switches using koopman operators. *Mathematics* **9**, 3133. (doi:10.3390/math9233133)
59. Villaverde AF, Barreiro A, Papachristodoulou A. 2016 Structural identifiability of dynamic systems biology models. *PLoS Comput. Biol.* **12**, e1005153. (doi:10.1371/journal.pcbi.1005153)
60. Sakib SA, Pan S. 2024 Learning noise-robust stable koopman operator for control with physics informed observables. *arXiv*. (doi:10.48550/arXiv.2408.06607)
61. Sakib SA, Pan S. 2024 Learning noise-robust stable koopman operator for control with Hankel DMD. *arXiv* (doi:10.48550/arXiv.2408.06607)
62. Sinha S, Nandanoori SP, Barajas-Solano DA. 2023 Online real-time learning of dynamical systems from noisy streaming data. *Sci. Rep.* **13**, 22564. (doi:10.1038/s41598-023-49045-w)
63. Yong SZ, Zhu M, Frazzoli E. 2017 Simultaneous input and state estimation for linear time-varying continuous-time stochastic systems. *IEEE Trans. Automat. Contr.* **62**, 2531–2538. (doi:10.1109/TAC.2016.2601688)
64. Curi S, Levy K, Krause A. 2019 Adaptive Input Estimation in Linear Dynamical Systems with Applications to Learning-from-Observations. In *Proceedings of the 2019 IEEE 58th Conference on Decision and Control (CDC)*. Nice, France. (doi:10.1109/cdc40024.2019.9029379)
65. Kitanidis PK. 1987 Unbiased minimum-variance linear state estimation. *Automatica* **23**, 775–778. (doi:10.1016/0005-1098(87)90037-9)
66. Khalil HK. 2002 *Nonlinear Systems*. 3rd edn. New Jersey: Prentice Hall.
67. Doerr A. 2012 (Protein) timing is everything. *Nat. Methods* **9**, 783. (doi:10.1038/nmeth.2120)
68. Khmelinskii A, Meurer M, Ho CT, Besenbeck B, Füller J, Lemberg MK, Bukau B, Mogk A, Knop M. 2016 Incomplete proteasomal degradation of green fluorescent proteins in the context of tandem fluorescent protein timers. *Mol. Biol. Cell* **27**, 360–370. (doi:10.1091/mbc.E15-07-0525)
69. Villaverde AF, Tsiantis N, Banga JR. 2019 Full observability and estimation of unknown inputs, states and parameters of nonlinear biological models. *J. R. Soc. Interface* **16**, 20190043. (doi:10.1098/rsif.2019.0043)
70. Gao S, Dai Y, Rehman J. 2021 A Bayesian inference transcription factor activity model for the analysis of single-cell transcriptomes. *Genome Res.* **31**, 1296–1311. (doi:10.1101/gr.265595.120)
71. Jamiree. 2025 Estimating-Varying-Parameters-in-Dynamical-Systems. Github. See <https://github.com/Jamiree/Estimating-Varying-Parameters-in-Dynamical-Systems>.